o

DS & Algo
-00940224-

- **Student 1:** [redacted] – [redacted]
- **Student 2:** Suheil Khourieh – [redacted]

Question 1:

Let $G = (V, E)$ be a directed graph with weight function $w: E \to \mathbb{R}_{\geq 0}$ on the edges and let $s \in V$ be any vertex.
Define

$$E' = \{(u, v) \in E | \exists P = \langle s, \dots, u, v \rangle, w(P) = \delta(s, v)\}$$

i.e. the edge $(u, v) \in E$ belongs to the set E' if there exists a shortest path from a vertex s to a vertex v that ends in the edge (u, v). Plan an algorithm that takes as an input a directed graph $G = (V, E)$ and a source vertex $s \in V$ and prints E'. The run-time complexity of the algorithm should be $O(nlog(n) + m)$. You should provide pseudocode and a verbal explanation on the way the algorithm works, prove correctness and analyze the time complexity.

Solution:

We have a directed graph with non-negative edge weights and a starting vertex s.
Our goal is to find all edges that are part of at least one shortest path from s to any other vertex. We need to do this efficiently in $O(nlogn + m)$ time.

Algorithm explanation:
We will start by running $Dijkstra's\ Algorithm$ to compute the shortest path distances $\delta(s, v)$ from s to all other vertices.
After that we will iterate over all edges (u, v) in E.
For every edge we will check if it's part of a shortest path by checking
If $d[u] + w(u, v) = d[v]$ and we will add it to E'.

Hence here is the pseudo code of the algorithm
$Edges\_in\_shortest\_path(G, w, s)$:
1: $Run\ Dijkstra(G, w, s)\ to\ compute\ shortest\ path\ distances\ d[v]\ for\ all\ v \in V.$
2: $for\ each\ edge\ (u, v) \in G.E\ do$:
3: $\quad if\ d[u] \neq \infty\ AND\ d[v] == d[u] + w(u, v)\ then$:
4: $\quad\quad Add\ (u, v)\ to\ E'$
5: $Print\ E'$

Time complexity analysis:

$Dijkstra's\ Algorithm$: $O(nlogn + m)$.
Iterating over all edges $O(m)$.
Hence time complexity is dominated by $Dijkstra's\ Algorithm$ and is $O(nlogn + m)$ as requested.

Correctness of algorithm:

Correctness of E' definition -Each edge $(u, v) \in E'(u, v)$ satisfies:
$d[v] = d[u] + w(u, v)$ meaning it is part of at least one shortest path.

$d[v]$ computed by Dijkstra guarantees correctness as we have seen in the $Dijkstra's$ tutorial lesson.

Question 2:

Let $G = (V, E, w)$ be a weighted directed graph with a weight function $w: E \to \mathbb{R}$ on the edges where there doesn't exist any negative cycles in the graph. Plan an algorithm that decides if there exists in the graph G a cycle of weight 0. The run time of the algorithm should be $O(mn)$. You should explain accurately and concisely how the algorithm works and prove it's correctness. You are not required to provide pseudocode.

Solution:

Algorithm explanation:

Since the graph may be disconnected, we first ensure that all nodes are reachable by adding a new source vertex s' and connecting it to every vertex in G with zero-weight edges, this is necessary for running $Bellman - Ford$.

By connecting all vertices to s' we will construct a new graph G' such that it follows the following rules:
$V' = V \cup \{s'\}$ (we add a new node s')
$E' = E \cup \{(s', v) \mid v \in V\}$
$w'(u, v) = w(u, v)\ \forall (u, v) \in E(u, v)$ (same edge weights as in G)
$w'(s', v) = 0\ \forall\ v \in V$ (connecting s' to every node with a zero-weight edge).

After creating the new graph G' we will run $Bellman - Ford$ on G' using s' as the source vertex and we will iterate over V' vertices.

After doing that we shall construct a new graph H such that $H(V, E_h)$
Where $E_h = \{ (u, v) \in E \mid d[v] = d[u] + w(u, v) \}$

After doing that we can run $DFS\ algorithm$ over E_h to determine whether there are any cycles withing E_h graph which will necessarily indicate a zero weight graph in G (our original graph) since E_h only leaves the zero weighted edges in it as requested .

A simple pseudocode for the algorithm that includes our main steps would look like this:

$Detect\_Zero\_Weight\_Cycle(G)$:
1. $Add\ a\ new\ node\ s'\ to\ G, connecting\ it\ to\ all\ v \in V\ with\ edges\ of\ weight\ 0.$
2. $Run\ Bellman - Ford(G', s') to\ compute\ shortest\ path\ distances\ d[v] \forall v \in V.$
3. $Construct\ H = (V, E_h), where$:
$E_h = \{(u, v) \in E \mid d[v] = d[u] + w(u, v)\}$
4. $Run\ DFS(H)\ to\ check\ for\ cycles$:
$- If\ a\ cycle\ is\ found, return\ "Zero - weight\ cycle\ detected"$.
$- Otherwise, return\ "No\ zero - weight\ cycle"$.

Time complexity analysis:

Constructing G' takes $O(n)$

Running $Bellman - Ford$ takes $O(nm)$ as shown in the lectures
Constructing H takes $O(m)$
Running DFS takes $O(n + m)$
Overall the time complexity does not exceed $O(nm)$ as requested.

Proof of correctness:

Adding s' ensures that all nodes are reachable as needed in Bellman-Ford algorithm.
Bellman-Ford finds the shortest paths correctly
The graph H contains only edges that belong to shortest paths.
If a cycle exists in H, it must be a zero-weight cycle because all edges in H preserve shortest path distances and the cycle will be detected by DFS algorithm as shown in lecture.

Question 3:

Within this question, assume that k directed graphs $G_1 = (V, E_1), G_2 = (V, E_2), \dots, G_k = (V, E_k)$ are given in the following way: the graph $G = (V, E = E_1 \cup \dots \cup E_k)$ is given using an adjacency list and for every edge $e \in E$, an array is given I_e of length k where for every $1 \leq j \leq k$ the following holds: $if\ e \in E_j\ then\ I_e[j] = 1, else\ I_e[j] = 0$. In addition, a wight function is defined $w: E \to \mathbb{R}$ where it is guaranteed that in the graph G there does not exist any cycles of a negative weight. For every vertices pair $u, v \in V$ and for every $1 \leq j \leq k$, we mark with $\delta_j(u, v)$ the distance (relatively to the weight function w) from u to v in the graph G_j.

You need to describe an algorithm that takes as an input k directed graphs $G_1 = (V, E_1), G_2 = (V, E_2), \dots, G_k = (V, E_k)$ with a weight function w as described above and a source vertex $s \in V$, the algorithm calculates the values $\delta_1(s, v), \dots, \delta_k(s, v)$ for every vertex $v \in V$. The run-time complexity is required to be $O\big(k \cdot n \cdot log(n) + m \cdot (n + k)\big)$ where $m = |E|$. You should prove the correctness of the algorithm and analyze the run-time. No need to provide pseudo-code.

Solution:

High-level idea:

We would love for the weight function to be non-negative, if that was the case we would run the optimized implementation of $Dijkstra's\ Algorithm$ using a Fibonacci heap for every graph (k times), but unfortunately this is not the case, hence we will try to reweigh all the edges making them non-negative while maintaining the shortest paths (A path is a shortest path before the reweighing if and only if it is a shortest path after the reweighing).

Reweighing the edges:

P is a shortest path connecting u with v in a directed weighted graph if and only if $w(P) \leq w(P'), \forall P'\ path\ connecting\ u\ with\ v$.

We can easily see that the only common thing between all paths connecting every pair of vertices u, v is the starting and ending vertices ($u\ and\ v$), other than that there isn't necessarily other common things such as the vertices visited in the paths, hence we would love to find a way to reweigh the edges of the path where the new weight of a path is dependent on the starting and ending vertex, thus, maintaining the shortest path property after the reweighing.

We would love to have a function $h(v), \forall v \in V$ that by applying it to $w(u, v)$ we will get a non-negative result.

Construct a new graph G':

Construct a new graph $G' = (V \cup \{S'\}, E \cup E')$ where $E' = \{(S', v), \forall v \in V\}$ and change the weight function so that $w(e) = 0, \forall e \in E'$, in other words, construct a new graph G' by adding a new vertex S' and connecting it to all the other vertices with direct edges of weight 0.

Running $BellmanFord's\ Algorithm$ on G' and save $\delta(S', v), \forall v \in V$ in a new array $h[v]$.

Observation (1): $h[v] \leq 0, \forall v \in V$.
This is true because of the way G' was constructed and because we ran the algorithm from the newly added vertex S'.

Observation (2): Since $h[v] = \delta(S', v), \forall v \in V$, based on the triangle inequality we know that $h[v] \leq h[u] + w(u, v) \Rightarrow 0 \leq w(u, v) + h[u] - h[v]$.
s
We have found a way to reweigh the edges making them non-negative, note that the shortest path remains a shortest path and no additional shortest paths are added, this is true because all the paths connecting u, v are changed by the same value.

Hence, by constructing a new graph G' as described above and running $BellmanFord's\ algorithm$ and storing the values in $h[v]$, then changing the values of the edges as described above we will get a graph that has no negative weights while maintaining the shortest path property between the vertices.
Removing the added vertex S' and all it's edges will leave us with an undirected connected graph with non-negative weights, this is the prefect scenario for the $Dijkstra's\ Algorithm$.

Back to the question, we have k undirected connected graphs with the modified weight function.
We will run $Dijkstra's\ Algorithm$ on every graph of the k graphs starting from the desired vertex S, but instead of saving $\delta_j(S, v), \forall 1 \leq j \leq k$ as an attribute for the vertices we will create a new $n \times k$ matrix that will hold the $\delta(S, v)$ for every graph and for every node in the following way, let $dist$ be the matrix:
$dist[v][j] = \delta_j(S, v) + \boldsymbol{h}[\boldsymbol{v}] - \boldsymbol{h}[\boldsymbol{S}], \forall v \in V, 1 \leq j \leq k$.

Quick look:
Here is a quick look on what we did, we reweighted the edges making them non-negative by running $BellmanFord's\ Algorithm$ the graph containing all the k graphs, ran $Dijkstra's\ Algorithm$ while saving the result after deducting what was added from the reweigh in a matrix.

Proof of correctness:
We have already proven above that the way we reweighted the edges will not result in unwanted new shortest paths and will keep the shortest paths before the reweighing shortest paths after the reweighing.
No need to prove anything else because it was given that the graph contains no negative-cycles.
We already saw a proof for both $BellmanFord's\ and\ Dijkstra's\ Algorithm$ and their correctness is derived from their proof.

Run-Time Complexity:
Step 1: Construct a new graph G' as described above, doing so is very straight forward and can be achived in $O(n + m)$.

Step 2: Run $BellmanFord's\ Algorithm$ on the new constructed graph is $O(m \cdot n)$

Step 3: Calculate the new weight for every edge in G is can be achieved by iterating over the old weigh function and updating each value as described above. This operation takes $O(n + m)$.

Step 4: Disconnect the newly added vertex S' and its edges, this can take $O(n + m)$ also.

Step 5: Construct a new matrix filled with $-\infty$, this takes $O(nk)$ (The size of the matrix).

Step 6: Run $Dijkstra's\ Algorithm$ from the given S for each sub-graph of the total k graphs and save the result in the new matrix from the previous step, each call for the $Dijkstra's\ Algorithm$ will take $O(nlog(n) + m)$ if we implement it using a Fibonacci heap as mentioned in the lectures, hence in total we get $O(knlog(n) + km)$.

Adding the results together we will get:

$$O(3 \cdot (n + m) + mn + nk + knlog(n) + km) = O\big(knlog(n) + m \cdot (n + k)\big)$$

Exactly as we were asked ■.

Question 4:

Given a directed graph $G = (V, E)$ with weight function $w: E \to \mathbb{R}$ on the edges and assume that there are no negative-weight cycles in the graph. In this graph, every vertex $v \in V$ is defined to be a $source$ vertex **or** a $target$ vertex. That is, there exists a partition of the vertices into two sets $V_1, V_2 \subseteq V$ where V_1 contains only the $source$ vertices and V_2 contains only the $target$ vertices and $V_1 \cap V_2 = \emptyset$. The type of a vertex v is represented by a field $v.type$.

Design an algorithm that computes for every $source$ vertex the distance to the nearest $target$ vertex. The time complexity of the algorithm should be $O(mn)$. Provide pseudocode and a verbal explanation of how the algorithm works, prove its correctness and analyze its running time.

Solution:

High-level idea:

We are asked to find the shortest path from every vertex in the $source$ set to any vertex in the $tager$ set, if the problem was reversed, this would be achievable by adding a virtual new vertex S by connecting it to every vertex in the $source$ set with zero weight edges and then running $BellmanFord's\ Algorithm$, resulting in a shortest distance for every vertex in the $target$ set that passes through a vertex in the $source$ set, because this is not the case, we can reverse the graph by reversing all the edges while maintaining their original weight, and then continue with the idea above, thus achieving the desired.

Lemma:

Given a shortest path P between two vertices u, v in a directed weighted graph with no negative cycles, where $w(P) = c$ (Can be ∞ if v is not reachable from u).
Constructing a new graph $G' = (V, E')$ by reversing the edges while maintaining the weights $E' = \{(v, u): (u, v) \in E\}, w'(v, u) = w(u, v)$ will not affect the shortest distance between any pair of nodes. Even more than that, for every path P of weight w_0 in the original graph G there exists a reversed path P' of the same weight w_0 in the reversed graph G'.
$P = \langle v_0, v_1, \dots, v_{k-1}, v_k \rangle \in G \Leftrightarrow P' = \langle v_k, v_{k-1}, \dots, v_1, v_0 \rangle \in G'$ and $w(P) = w'(P') = c$.

Proof:

Let $P = \langle v_0, v_1, \dots, v_{k-1}, v_k \rangle$ be a path in $G = (V, E)$, by definition of G', every edge $(u, v) \in E \Leftrightarrow (v, u) \in E'$, hence, for every edge $e = (v_i, v_{i+1}) \in P$ there exists a mirrored edge $e' = (v_{i+1}, v_i) \in E'$, thus it is possible to construct P'.
$P \in G \Leftrightarrow P' \in G'$, this contains the fact that no new paths will be added.
We are left to prove that $w(P) = w'(P')$, we have already proven that for every path in G there exists a mirrored version of it in G', we know that the mirrored path is achieved by mirroring the edges in the original path, thus:
$w'(P') = \sum_{e' \in P'} w'(e') = \sum_{e \in P} w(e) = w(P) \Rightarrow w(P) = w'(P')$. ■

After we have proven the lemma, we can construct a new directed weighted graph $G' = (V', E')$ that contains no cycles by adding a new vertex S to the original set of vertices V, connecting it to every vertex in the set V_2 (the $target$ set) using a zero

weight edge and by reversing all the edges in E while maintaining their original weight, hence defining a new weight function $w'(e), \forall e \in E'$.

$$V' = V_1 \cup V_2 \cup \{S\}, E' = \{(v,u): (u,v) \in E\} \cup \{(S,v): v \in V_2\}$$
$$w'(v,u) = w(u,v), \forall (u,v) \in E$$

After we are done constructing the new graph $G' = (V', E')$, we will run $BellmanFord's\ Algorithm$ on G' starting from S, the result of running this is the distance of a shortest path from S to all the other vertices in the graph, in our case, since S has the following edges $(S,v), \forall v \in V_2$, the shortest path from S to every vertex in V_1 (the $source$ vertices) must pass through a vertex from the $target$ set and the path from S to any vertex in V_2 is of weight zero, hence it will **not** affect the length of the shortest path.

Pseudocode:

```
1   procedure reverseEdgesAndWeights(E, w) {
2       ReversedE = new empty edge set
3       newW = new empty weight function
4       for all (u,v) in E do
5           reverseEdges.append((v,u))
6           newW.add((v,u), w(u,v))
7       end
8       return ReversedE, newW
9   }
```

```
11  procedure addZeroWeights(V, S){
12      E = new empty edge set
13      w = new empty weight function
14      for all v in V do
15          E.append((S,v))
16          w.add((S,v), 0)
17      end
18      return E, w
19  }
```

This is the pseudocode for the helper methods used:
1) $reverseEdgesAndWeights(E,\ w)$, this procedure takes as an input an edges set E and a weight function w and returns a reversed version of E and an adjusted version of the weight function, it does so by iterating over all the edges in the edges set E and appending the mirrored version of the edge in the new set, it then adds the weight of the newly added edge to be the same as the original (before the mirroring). When done with the loop, the values are returned.
The run-time of this procedure is clearly $O(m)$ where m is the number of edges in E.

2) $addZeroWeights(V,\ S)$, this procedure takes as an input a set of vertices V and a single vertex S and returns a new set of edges and weight function where the set contains edges connecting S with every vertex in V using an edge of weight zero.
It does so by iterating over all the vertices in V, appending the new edge to the new edges set and adds the weight zero to the weight function. When the loop terminates, the new set of edges E and the new weight function are returned.
The run-time of this procedure is $O(n)$ where n is the number of vertices in the set V.

```
21  procedure sourceToTargetDistance(G, w) {
22      V' = G.V + S
23      E', w' = reverseEdgesAndWeights(G.E, w)
24      V_2 = empty vertex set
25      for all v in V do
26          if v.type == target do
27              V_2.append(v)
28          end
29      end
30      E', w += addZeroWeights(V_2, S)
31      G' = new graph(V', E')
32      Bellman_Ford(G', w', S)
33  }
```

This procedure will take care of everything we were asked, this procedure takes as an input a graph $G = (V, E)$ and a weight function w where the set of vertices V is assembled from two sets of vertices V_1, V_2, where $V_1 \cap V_2 = \emptyset \; and \; V_1 \cup V_2 = V$ as described in the question.
The procedure first constructs a new graph $G' = (V', E')$ as described above, it does so by adding a setting V' to be the old set of vertices with the additional set S, it then creates E', w' by calling the helper procedure $reverseEdgesAndWeights$ that was introduced above, it then extracts the set V_2 by iterating over all the vertices and adding the relevant vertices to V_2. It then updates E', w using the helping procedure $addZeroWeights$ which results in the graph G' exactly as we wanted.
Finally, a call to the known $Bellman\_Ford\ Algorithm$ is done from S which results in finding the shortest length from S to all the other vertices, including the subset V_1 which will achieve the asked in the question.

Proof of correctness:
We have already proven a lemma earlier that will help us prove the correctness of the algorithm, we know that what we did 100% did not affect the length of the shortest distance between two nodes, to further explain, we know that:

$$\delta_G(u, v) = \delta_{G'}(v, u), \forall u, v \in E \; or \; E'$$

Hence, we know that $\delta_G(u, v) = \delta_{G'}(v, u), \forall u \in V_1, \forall v \in V_2$.
We have found $\delta_{G'}(S, u), \forall u \in V_1$, we are left to prove that

$$\exists v \in V_2 : \delta_{G'}(S, u) = \delta_G(u, v), \forall u \in V_1$$

Let P' be the shortest path from S to u, we know based on how G' is constructed that P' will look something like this: $P' = \langle S, v, \dots, u \rangle$ where $v \in V_2$ because we have forced S to pass through some vertex in V_2 before proceeding to the graph. Based on the newly adjusted weight function w', we know that $w'\big((S, v)\big) = 0, \forall v \in V_2$, hence,

$$w'(P') = w'(\langle v, \dots, u \rangle)$$

Based on the lemma proven before $w'(P') = w(P)$, for every path P and it's mirrored version $P' \Rightarrow w'(\langle v, \dots, u \rangle) = w(\langle u, \dots, v \rangle) \Rightarrow \delta_{G'}(S, u) = \delta_G(u, v)$ and this describes exactly what we are looking for. ■

Runtime complexity:

Running the procedure $sourceToTargetDistance$ is $O(nm)$ where n and m are the number of vertices and edges respectively in the graph taken as an input:
Assembling G' takes $O(n+m)$, no need to prove this since we have already shown that assembling the G' is done using helper methods where we have shown their time complexity, the loop line 25-29 takes $O(n)$, this loop might be unnecessary depending on the representation of the given graph G, but preforming the loop won't affect our total time complexity, hence we will do it.
Finally, running $Bellman\_Ford's\ Algorithm$ takes $O(mn)$ according to the lectures, hence in total we have $O(mn+m+n) \Rightarrow O(mn)$. ■

Question 5:

Let $G = (V, E)$ be an undirected connected graph with a weight function $w: E \to \mathbb{R}_{>0}$ that assigns a positive weight $w(e) > 0$ for every edge $e \in E$.
Answer the following parts. Note that the given in every part is only relevant to the part itself.

Part 1:

For a pair of vertices $u, v \in V$, let $P = \langle u, \dots, v \rangle$ be a simple shortest path (relative to the weight w) between u and v in G and denote the number of edges in P with k. Let T be an MST of G, and mark the simple unique path from u to v in T with $P_{u,v}$.
Prove/Disprove: The number of edges in $P_{u,v}$ is at least k.

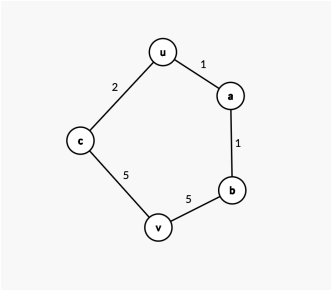
Solution:

The claim is incorrect!

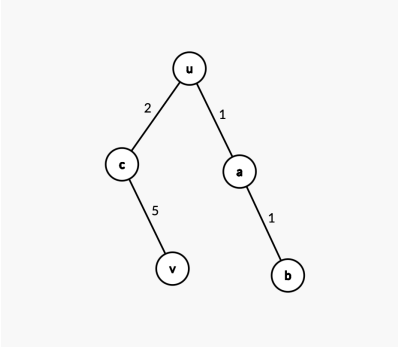
Disprove the claim using a counter example:
Let $V = \{u, a, b, c, v\}, E = \{(u, a), (a, b), (b, v), (v, c), (c, u)\}$ and $G = (V, E)$ be an undirected connected graph with the following weight function:

$$w(e) = \begin{cases} 1, & if\ e \in \{(u, a), (a, b)\} \\ 2, & if\ e \in \{(c, u)\} \\ 5, & if\ e \in \{(b, v), (v, c)\} \end{cases}$$

Here is the graph G with the given weights:



$P = \langle u, a, b, v \rangle$ is a shortest path relative to the weight function connecting u with v, the number of edge in P is $k = 3$.
Let T be the MST we got from running an algorithm that finds MST, for example $Prim's\ Algorithm$ that we saw in class.
Running $prim's$ on the graph G can result in two different $MSTs$, but we are interested in the following MST:



We can easily see that this indeed is an MST of G.

The path connecting u with v in T is $P_{u,v} = \langle u, c, v \rangle$ where the number of edges in $P_{u,v}$ is $j = 2$.

$$j < k \Rightarrow 2 < 3 \Rightarrow \textit{The claim is incorrect} \blacksquare$$

Part 2:

For a pair of vertices $u, v \in V$, let $P\langle u, \dots, v\rangle$ a simple path between u and v in G, and denote the number of edges in P with k. Let T be a MST of G.

Prove/Disprove: If $w(P) > k \cdot \delta(u, v)$ then $P \nsubseteq T$.

Solution:

The claim is correct!

Assume by contradiction that the claim is incorrect, i.e. there exists a graph G as described in the question that contains a simple path P of length k that connects u with v, where $P \subseteq T$.

Before proceeding with the proof, let's denote the following:

1. $P' = \langle u, \dots, v\rangle, w(P') = \delta(u, v)$.($P'$ is any shortest path in G that connects u with v).
2. Let e_1 be an edge in P where $w(e_1) \geq w(e'), \forall e' \in P$. In other words, let e_1 be the/a heaviest edge in P. $\Rightarrow e_1 = \text{argmax}_{e \in P}\, w(e)$.
3. Similarly, let e_2 be an edge in P' where $w(e_2) \geq w(e'), \forall e' \in P'$.

In other words, let e_2 be the/a heaviest edge in P', or even between all the shortest paths connecting u and v. $\Rightarrow e_2 = \text{argmax}_{e \in P'}\, w(e)$.

Our assumption states that $\exists simple\ path\ P\ of\ length\ k\ that\ connects\ u\ with\ v \subseteq G :$ $w(P) > k \cdot \delta(u, v)$ **and** $P \subseteq T\ where\ T\ is\ an\ MST\ of\ G$.

Before proceeding, let's prove the following lemma:

Lemma: Let P_1, P_2 be two simple paths connecting u with v in an undirected connected graph G and let T be an MST of G.

$P_1 \subseteq T \Rightarrow e_1 \leq e_2\ where\ e_1, e_2\ are\ the\ heaviest\ edges\ in\ P_1, P_2\ respectively$

Proof: Assume by contradiction that the claim is incorrect, i.e. $P_1 \subseteq T\ and\ e_1 > e_2$. Finding an MST is done using a greedy search, hence it searches for the lightest edges possible, thus it will discover e_2 before e_1 since $e_1 > e_2$, not only that, but all the edges of the path P_2 will be discovered before e_1 because e_2 is the heaviest among them, resulting in the path P_2 to be included in T and not P_1 since only one of them can be included. This contradicts our assumption that $P_1 \subseteq T$. ($\rightarrow\leftarrow$) The lemma is correct!

After we have proven the lemma, based on it we know that if $P \subseteq T$ then $e_1 \leq e_2$. Indicating that the heaviest edge in P is lighter than the heaviest edge in P'.

We can easily see that $w(P) \leq k \cdot e_1$ and based on the lemma proven above and the fact that $e_1 \leq e_2$ we get that $w(P) \leq k \cdot e_2$

Based on our assumption we know that $k \cdot \delta(u, v) < w(P)$.

Hence, $k \cdot \delta(u, v) < w(P) \leq k \cdot e_2 \Rightarrow k \cdot \delta(u, v) < k \cdot e_2$

We also know that $e_2 \leq \delta(u, v)$, connecting everything together we get:

$k \cdot \delta(u, v) < k \cdot e_2 \leq k \cdot \delta(u, v) \Rightarrow k \cdot \delta(u, v) < k \cdot \delta(u, v) \Rightarrow (\rightarrow\leftarrow)$. ■

Thus, the claim in the question is indeed correct!